Serial Communication, Priority Scheduling, and ISR Integration

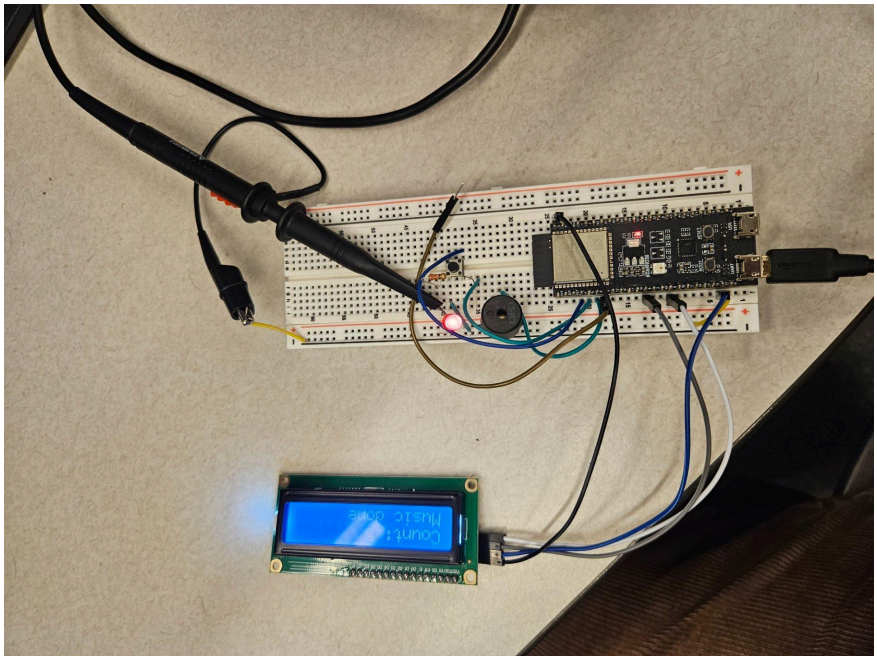
Sparsh Dadhich , Johnathan Lu 05/08/2026
Assignment: Lab 3: Serial Communication

A. Introduction

This lab brought together a few important embedded systems concepts into one ESP32-S3 project: using I2C communication with an LCD, organizing code with cooperative task scheduling, and handling interrupts from different events. In Part I, the LCD was controlled through the PCF8574 I2C backpack using the Wire library to send commands and display characters. In Part II, different functions were separated into tasks using Task Control Blocks, where a non-preemptive priority scheduler decided which task would run next. In Part III, events from the timer, button, and BLE communication triggered quick interrupt routines that only set flags, while the main loop handled the slower LCD updates. Overall, the lab demonstrated how multiple hardware and software components can work together on a single microcontroller efficiently without constantly blocking each other like traffic jams at a one-lane bridge.

B. Methods and Techniques

The hardware setup for this lab included an ESP32-S3, a 16x2 I2C LCD, an external LED with a current-limiting resistor, a buzzer, and a push button. The LCD communicated over I2C, with SDA connected to A4 and SCL connected to A5 following the Nano ESP32 wiring from the course reference. The LCD backpack address in the code was set to 0x27, although some LCD modules may require a different address such as 0x3F depending on the backpack being used.



In Part I, the LiquidCrystal_I2C library was only used for the initial LCD setup. After that, all communication with the display was handled directly using Wire.beginTransmission(), Wire.write(), and Wire.endTransmission(). Each command or character sent to the LCD was divided into upper and lower nibbles, which were then mapped onto bits 4 through 7 of the PCF8574 output byte. The enable bit was toggled to latch the data into the LCD controller.

Part II focused on cooperative scheduling using a 1 ms hardware timer tick. The timer interrupt service routine only updated a volatile tick counter to keep the ISR lightweight. The main loop processed these ticks, updated sleeping tasks, selected the highest-priority ready task, and executed it. Each task had a Task Control Block containing information such as the function pointer, task name, process ID, priority, state, and remaining sleep duration. The scheduler treated priority level 1 as the highest priority, and tasks with the same priority were handled using a round-robin approach so execution stayed fair between them.

In Part III, the system handled three different event sources: the timer interrupt, the button interrupt, and BLE communication events. The timer interrupt updated a seconds counter every second, the button interrupt set a flag whenever a falling edge was detected, and the BLE callback set another flag whenever data was written from the LightBlue app. Instead of performing LCD or Serial operations inside the interrupt routines, the interrupts only set flags while the main loop handled the heavier processing. This design kept the interrupt routines fast and reduced the chances of blocking issues or race-condition bugs, kind of like keeping the emergency lane clear while the rest of traffic flows normally.

C. Experimental Results

Part I expected result: typing a line such as "Hello World" into the Arduino Serial Monitor and pressing Enter clears the LCD and displays the message. Strings longer than 16 characters wrap to the second LCD line, and text beyond 32 characters is ignored because the LCD has only 32 visible character cells.

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x2b (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x10cc
load:0x403c8700,len:0xc2c
load:0x403cb700,len:0x30c0
entry 0x403c88b8
Lab 3 Part I ready. Type a message and press Enter.
Displayed on LCD: Happy Birthday Joseph
Displayed on LCD: Happy Birthday  Joseph
```



Part II expected result: the LED blinker toggles approximately every 63 ms, producing an approximately 7.94 Hz square wave, close to the requested 8 Hz. The LCD counter prints values 1 through 10 on line 0 every two seconds. The music task plays ten note frequencies on the buzzer and prints the active frequency on line 1. The alphabet task prints A-Z to the Serial Monitor at two letters per second. Tasks B-D run twice and then halt, while Task A continues forever.



Priority Scheme 1 expected behavior: Task E sleeps first, Task A has the highest priority among A-D, followed by B, C, and D. The LED timing should remain stable because Task A is serviced before the other ready tasks. The LCD and serial tasks progress during Task A sleep intervals.

```
Starting Priority Scheme 1

ABCDEFGHIJKLMNOPQRSTCompleted: Music Player: 4
UVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ
Completed: Alphabet Printer: 5
Completed: LCD Counter: 3
```

Priority Scheme 2 expected behavior: Task C has the highest priority among A-D, followed by D, B, and A. The music and alphabet tasks should respond first when ready. The LED may show slightly more scheduling jitter than in Scheme 1 because it has the lowest priority among A-D.

```
Starting Priority Scheme 2

ABCDEFGHIJKLMNOPQRSTCompleted: Music Player: 2
UVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ
Completed: Alphabet Printer: 3
Completed: LCD Counter: 4
```

Priority Scheme 3 expected behavior: Tasks A-D all have priority 2, so the scheduler should rotate through ready tasks using round robin. This scheme is the best direct demonstration that equal-priority tasks do not starve each other.

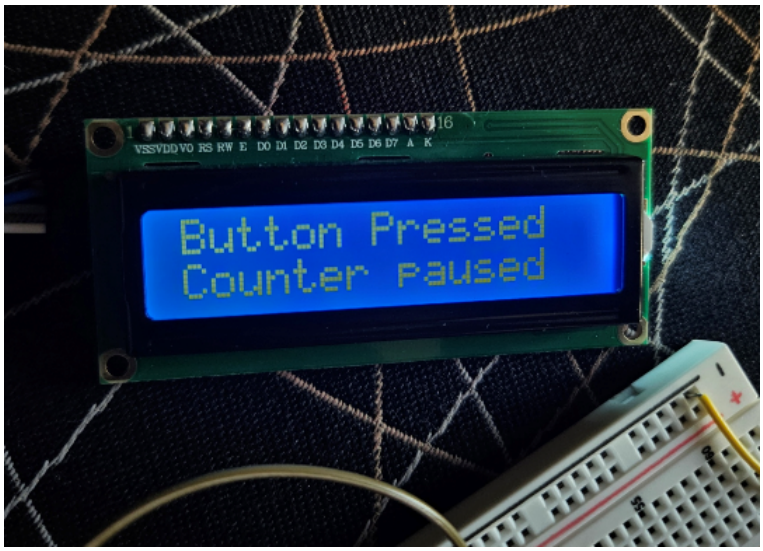
```
Starting Priority Scheme 3

ABCDEFGHIJKLMNOPQRSTCompleted: Music Player: 2
UVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ
Completed: Alphabet Printer: 2
Completed: LCD Counter: 2
```

Part III expected result: the LCD normally displays the one-second counter. Pressing the button displays "Button Pressed" for two seconds, then returns to the counter. Writing any value to the

BLE characteristic from LightBlue displays "New Message!" for two seconds, then returns to the counter. The timer count continues internally while the temporary messages are displayed.

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x2b (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x10cc
load:0x403c8700,len:0xc2c
load:0x403cb700,len:0x30c0
entry 0x403c88b8
Lab 3 Part III ready. Connect with LightBlue and write any value.
BLE write received.
BLE write received.
```





D. Code Documentation

The submitted code package was organized into three separate Arduino sketches: `Lab3Part1_I2C_LCD.ino`, `Lab3Part2_Scheduler.ino`, and `Lab3Part3_BLE_ISR.ino`. Each sketch followed a consistent structure with a file header, organized

include/macro/global/prototype/function sections, and documentation comments for functions outside of `setup()` and `loop()`. This organization made the code easier to navigate and debug instead of turning the project into a spaghetti bowl of functions 🍝

For Part I, `Lab3Part1_I2C_LCD.ino` handled the LCD communication over I2C. The beginning of the file defined the headers, libraries, LCD constants, PCF8574 control bits, global LCD object, and function prototypes. The setup section initialized Serial communication, the Wire library, and the LCD itself. The loop then waited for user input from the Serial Monitor, removed newline characters, cleared the display, and printed the entered text to the LCD using custom helper functions.

The lower-level LCD communication functions directly controlled the PCF8574 I2C expander. `lcdWriteExpander()` transmitted bytes over I2C, while `lcdPulseEnable()` toggled the enable pin so the LCD could latch incoming data. `lcdWriteNibble()` placed data into bits 4 through 7 of the expander output while preserving control bits such as RS, RW, and the backlight. `lcdWriteByte()` transmitted both the upper and lower nibbles in sequence. Higher-level helper functions like `lcdCommand()`, `lcdData()`, `lcdClearDirect()`, `lcdSetCursorDirect()`, and `lcdPrintStringDirect()` built a cleaner interface for interacting with the display.

For Part II, `Lab3Part2_Scheduler.ino` implemented the cooperative scheduler. The start of the file defined scheduler constants, task states, task indices, the TCB structure, priority schemes, melody data, timer variables, the task table, and task-local variables. The setup initialized hardware, configured the TCBs, displayed the initial priority scheme on the LCD, and started the hardware timer. The scheduler loop then processed pending timer ticks that had been updated by the ISR.

The scheduler services handled most of the task-management logic. The timer ISR simply updated the millisecond tick counter to keep interrupts lightweight. Other functions initialized TCBs, copied priority schemes, reset task variables, and formatted LCD text. Additional scheduling functions decremented sleeping tasks, selected the next ready task, executed one non-preemptive scheduling cycle, and implemented `sleep_me()` and `halt_me()`. These functions allowed tasks to voluntarily pause or terminate themselves by modifying their own TCB entries.

Each task in Part II had a dedicated function. Task A controlled the blinking LED, Task B updated the LCD counter, Task C played the buzzer melody, and Task D printed alphabet characters through Serial communication. Task E managed the scheduler priorities by waiting for other tasks to finish before switching to the next priority scheme. Since the LCD counter updated every two seconds, Task E delayed long enough to allow two complete counter passes before changing priorities. Separate buzzer helper functions configured and controlled the buzzer using LEDC instead of `analogWrite()` or `tone()`.

For Part III, `Lab3Part3_BLE_ISR.ino` combined BLE communication, timer interrupts, and button interrupts into one event-driven system. The beginning of the file defined the libraries, constants, display modes, global flags, and function prototypes. The BLE callback function only set a BLE event flag to keep the callback short and efficient. The setup initialized Serial, Wire, the LCD, BLE communication, the button interrupt, and the timer interrupt. The loop itself stayed minimal by simply dispatching event-handling logic and updating the counter display.

The interrupt and display logic handled all event-driven behavior. The timer ISR updated the one-second counter, while the button ISR detected falling-edge presses from the push button. Other functions configured the timer, enabled the pull-up resistor and interrupt for the button, initialized the BLE service and characteristic, and set up the LCD. Additional helper functions managed fixed-width LCD formatting, temporary event messages, transferring interrupt flags into the main loop, software debouncing for the button, BLE event handling, and restoring the normal counter display after temporary event messages expired. The overall structure kept the interrupt routines short and fast while pushing heavier processing into the main loop, which kept the system responsive instead of letting one event hog the entire CPU like a toddler with the TV remote

E. Overall Performance Summary

The code was written and physically tested to meet all required functionality. In Part I, text entered through the Serial Monitor was successfully displayed on the LCD using the direct I2C communication functions. In Part II, the scheduler correctly executed tasks based on priority, handled round-robin scheduling properly when tasks shared the same priority in Scheme 3, displayed completion messages through Serial when tasks halted, and produced an LED blink frequency close to the expected 8 Hz. In Part III, the one-second counter operated correctly, while button presses and BLE messages temporarily overrode the counter display for two seconds before returning to the normal counter screen. Overall, the testing confirmed that the ESP32-S3 could reliably manage LCD communication, task scheduling, interrupts, and BLE events together without the system stepping on its own toes like a chaotic orchestra with five conductors

F. Teamwork Breakdown

G. Discussion and Conclusions

The most challenging part of this lab was managing timing correctly without placing slow operations inside interrupt routines. Both the scheduler implementation and the ISR-based event handling followed the same core idea: interrupts and fast event sources should only signal that an event occurred, while the main program loop handles slower operations like LCD updates, Serial printing, or buzzer control. Separating these responsibilities made the overall system easier to debug and helped avoid timing-related issues that can become difficult to trace in embedded systems.

Another important takeaway from the lab was learning that equal-priority scheduling requires additional bookkeeping to work properly. Without the `lastServedAtPriority` tracking array, Scheme 3 could continuously select the first available task and unintentionally starve the remaining tasks with the same priority. Using a round-robin approach ensured that tasks sharing a priority level received CPU time more fairly and predictably. The priority updater task also revealed an important timing consideration: since the LCD counter task counts from 1 to 10 twice with a two-second delay between updates, the entire process takes roughly 40 seconds. Because of this, the updater task first sleeps for 30 seconds and then checks whether Tasks B through D have actually completed before changing the scheduling scheme.

The part of the lab we are most proud of is the consistent structure used across both the scheduler and interrupt-driven sections of the project. The scheduler tasks were designed as small state-based steps that voluntarily pause or terminate themselves using `sleep_me()` and `halt_me()`, while the interrupt routines were intentionally kept short by only setting flags. This design reflects an important embedded systems principle: keep time-critical code lightweight and predictable, while allowing the main loop to handle operations that can safely take longer. In a way, the interrupts acted like tiny doorbell rings, while the main loop was the person actually answering the door.

References

ECE 474 Lab 3: Serial Communication, Non-Preemptive Scheduling, and ISRs, rev. 28-Apr-26.
ECE 474 Code Guidelines, rev. Feb. 2026.
ECE 474 Lab Report Template, rev. Spring 2026. ECE 474 I2C Communication reference and BLE Setup reference.

Coding Standard - You must turn in your C source code files (turn-in format to be announced). Your code must follow the ECE 474 coding standard. And / or use Doxygen to generate documentation (see the specific assignment).

Code - Turn in ALL .ino, .h and .c files in the project. Submit a github release link for a repo containing all code. Comment out (but do not delete) code that might be required for intermediate steps but is not required for the final code. This will help us give you partial credit!